

Funktionale Programmierung mit Python

Can Nayci

09.11.2023

- 1 Vorab
 - Über Python
- 2 Sprachkonzepte
 - Iteratoren
 - Generatoren
 - Listcomp's & Genexp's
 - Lambda & Currying
 - Einige Funktionen
- 3 Module
 - *itertools*
 - *functools*
 - *Caching*

Über Python

Was ist Python? [4]

- general-purpose / multiparadigmatisch
- Fokus auf leichter Lesbarkeit
- Ziel: wenig Code, große Wirkung
- Kaum etwas, das nicht unterstützt wird

Über Python

C entry.c > ...

```
1  #include <stdio.h>
2
3  √ int main()
4  {
5      int x = 5;
6      printf("%d", x);
7
8      return 0;
9  }
10
```

(a) Beispielprogramm in C

Python entry.py > ...

```
1  x=5
2  print(x)
3
```

(b) Beispielprogramm in Python

Kurzer Vergleich

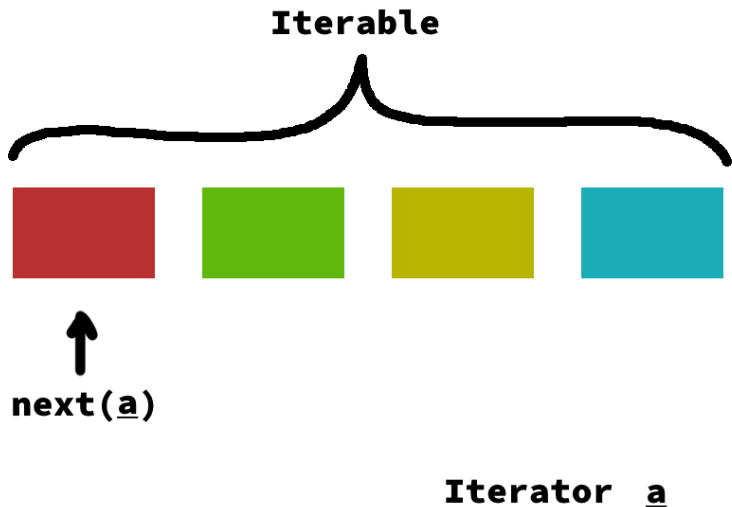
Über Python

- Funktionale Programmierung:
 - Problem aufgeteilt in Funktionen
 - Funktionen berechnen für Input → Output
 - für gleichen Input → gleichen Output!
- Sprache deckt FP bedingt ab → Module nötig
- Docs (über FP) sind sehr gut [1]

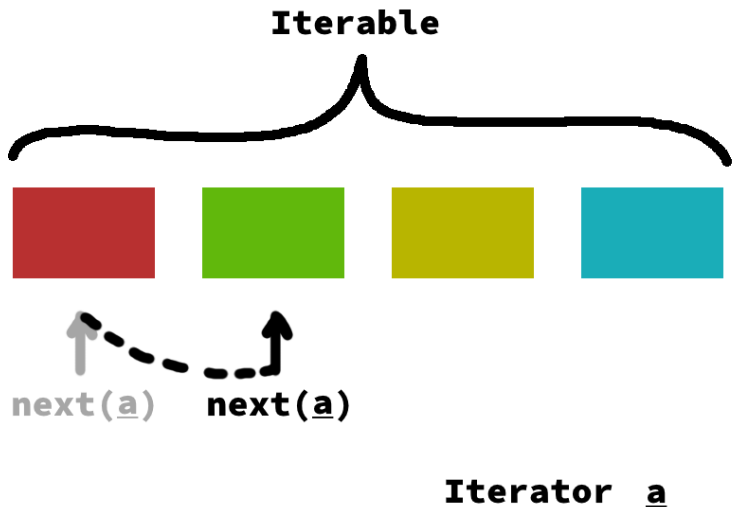
Iteratoren

- im Standard implementiert (v2.2)
- Daten als Strom
- Ähnlich zu anderen Sprachen
- Iteriert über *Iterable* mit *next(a)*, wenn *a* Iterator
- Interne Organisation:
 - Individuell
 - Kann Array-artig sein (Pre-Allocated)
 - Kann iterativ berechnet werden

Iteratoren



Iteratoren



Iteratoren

Code-Beispiel

iter.py

Generatoren

- im Standard implementiert (v.2.6?)
- Funktionsaufruf als Strom
- Generatorfunktion gibt Generator zurück
- Sei a ein Generator:
 - läuft mit $next(a)$ zum nächsten *yield* x für ein x
 - gibt Wert x zurück
 - speichert Funktionsinternes (lokale Variablen etc.)
 - läuft mit nächstem $next(a)$ zum nächsten *yield*

Generatoren

```
generator_func():
```

```
    some code...
```

```
    yield 13
```

```
    some code...
```

```
    yield 14
```

```
    some code...
```

```
    yield 15
```

```
    some code...
```

```
    yield 16
```

Generator a

Generatoren

```
generator_func():
```

```
    some code...
```

```
    yield 13
```

```
    some code...
```

```
    yield 14
```

```
    some code...
```

```
    yield 15
```

```
    some code...
```

```
    yield 16
```

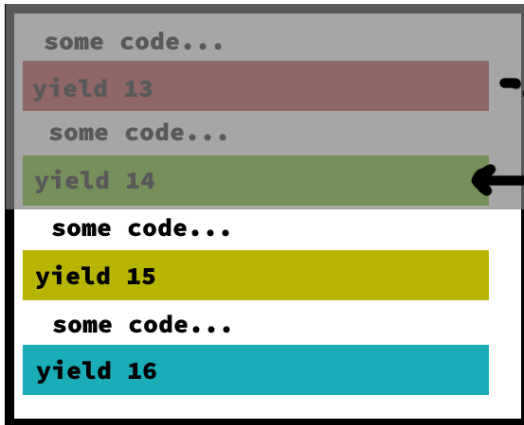
executed

next(a)
= 13

Generator a

Generatoren

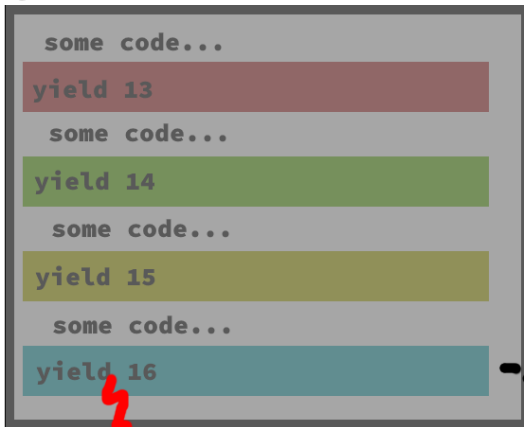
generator_func():



Generator a

Generatoren

generator_func():



executed



StopIteration



next(a)

Generatoren

Code-Beispiel

gen_lazy.py

Code-Beispiel

gen.py

- Speichereffizienz über Lazy-Evaluation

Code-Beispiel

real_lazy_problem.py

Generatoren

■ Kommunikation mit Generatoren

- *yield* *x* kann zu einer Variable hinzugewiesen werden
- Variable *None*, bis man Wert an Generator sendet
- *a.send(value)* mit Generator *a* und Wert *value*
- *send*-Funktion wie *next()*, lässt weiterlaufen

Code-Beispiel

gen_comm.py

Listcomp's & Genexp's

- Bekanntes Konzept aus anderen Sprachen, z.B. Haskell
- (*expr* for *i* in *iterable_data*) - generator expression
 - gibt Generator zurück
- [*expr* for *i* in *iterable_data*] - list comprehension
 - gibt Liste zurück
- Geht auch komplexer:
 - [*expr* for *x* in *iterable_data1* for *y* in *iterable_data2*]...

Code-Beispiel

listcomp_genexp.py

Lambda & Currying

- Lambda-Ausdrücke im Standard implementiert
- Anonyme Funktionen wie aus anderen funktionalen Sprachen
- Currying ermöglicht Funktionen höherer Ordnung
- Funktionen als Argumente
- Innere Funktionen, Funktionen von Funktionen
- Funktionen mit mehreren Argumentklammern

Code-Beispiel

lambda_currying.py

Mehr lesen → [3]

Einige Funktionen

- Funktionen wie Map, Filter...
- Meist Funktionsanwendung auf Iterierbare Objekte
- Geben ein Ergebnis oder überarbeitetes Iterable

Code-Beispiel

builtin_funcs.py

Mehr lesen → [3]

itertools

- Modul mit zahlreichen Funktionen über Iteratoren
- ermöglicht vorgefertigte Iterationsfunktionen
- Zusammenfassen von Iteratoren
- Kombinatorik / Permutation von Iteratoren etc...

Code-Beispiel

itertools_mod.py

functools

- Modul rund um Funktionen
- Arbeit mit Funktionen Höherer Ordnung
- Setzt (End-)rekursive Konzepte aus FP-Sprachen um
- u.a. Funktionsausführung auf Iteratoren leichtgemacht

Code-Beispiel

`functools_mod.py`

Mehr lesen → [2]

Caching

- Caching von Funktionskontexten
- Problem bei Rekursion: Gleiche Berechnungen immer wieder (umsonst) ausgeführt
- Problem gelöst durch Caching
 - Zwischenspeicherung von Endergebnissen
- *lru_cache* für Funktionen
- *cached_property* bei Klassenattributen

Code-Beispiel

caching.py

Mehr lesen → [2]

Quellen



AM Kuchling.

Functional programming howto.

Python v2. 7.2 documentation, 2013.



David Mertz.

Functional Programming in Python.

O'Reilly Media, 2015.



Guido Van Rossum and Fred L Drake Jr.

Python tutorial, volume 620.

Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.



Guido Van Rossum et al.

Python programming language.

In *USENIX annual technical conference*, volume 41, pages 1–36.
Santa Clara, CA, 2007.